

Tarea 3

Polars vs Pandas: Pipeline de Datos y Machine Learning

Curso	Computación Paralela
Profesor	Johansell Villalobos Cubillo
Autor	David Mora V.
Dataset	2015 Flight Delays and Cancellations (USDOT)

Descripción del dataset

Se utilizó el conjunto **2015 Flight Delays and Cancellations** del U.S. Department of Transportation (licencia CC0), obtenido desde Kaggle. Contiene **5,819,079 registros** de vuelos comerciales en EE. UU. durante 2015, distribuidos en tres archivos: **flights.csv** (~592 MB, 31 columnas), **airlines.csv** y **airports.csv** (tablas de referencia).

Problema: clasificación binaria. Se predice si un vuelo sufrirá un retraso significativo en la llegada, definido como **ARRIVAL_DELAY > 15 minutos** (umbral estándar del DOT). La variable objetivo **RETRASADO** está desbalanceada: aproximadamente un 18% de la clase positiva. Los vuelos cancelados o desviados (sin retraso de llegada definido) se eliminaron.

Pipeline implementado

Análisis exploratorio (Polars): estadísticas descriptivas, conteo de valores faltantes, distribución de variables, matriz de correlaciones y visualizaciones. Se identificó que **DEPARTURE_DELAY** presenta una correlación muy alta con el retraso de llegada.

Ingeniería de características (Polars): filtrado de cancelados/desviados, eliminación de columnas con fuga de datos (causas de retraso y mediciones post-vuelo conocidas solo después de aterrizar), manejo de nulos, y creación de nuevas variables: HORA_SALIDA, TRIMESTRE, ES_FIN_SEMANA y PERIODO_DIA. Se aplicó un **join** con airlines.csv (nombre de aerolínea) y una agregación **group_by** por aeropuerto de origen (VUELOS_ORIGEN, proxy de congestión). El dataset resultante: **5,714,008 filas × 19 columnas**.

Resultados de Machine Learning

Se entrenaron tres modelos con división estratificada 80/20 y manejo del desbalance (class_weight balanceado). Para Gradient Boosting se usó HistGradientBoostingClassifier, óptimo para datasets grandes.

Modelo	Tiempo (s)	Accuracy	F1	AUC
Regresión Logística	17.69	0.9257	0.8120	0.9653
Random Forest	301.98	0.9436	0.8448	0.9701
Gradient Boosting (Hist)	126.47	0.9314	0.8242	0.9719

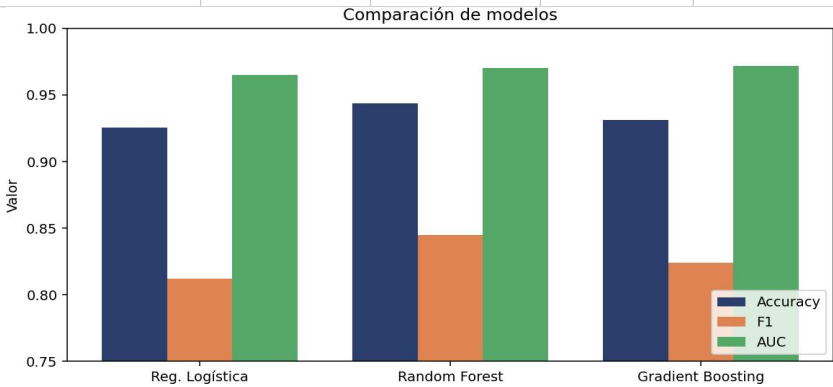


Figura 1. Comparación de métricas por modelo.

El **Gradient Boosting** obtuvo el mejor AUC (0.9719) con la mitad del tiempo que Random Forest. Random Forest lidera en accuracy y F1 pero es el más costoso (~5 min). La Regresión Logística es notablemente competitiva (AUC 0.9653) siendo 17× más rápida, lo que confirma que el retraso de salida aporta una señal casi linealmente separable. Variables más importantes: **DEPARTURE_DELAY**, SCHEDULED_TIME, TAXI_OUT, DISTANCE y HORA_SALIDA.

Benchmark Polars vs Pandas

Se implementó el **mismo pipeline** en ambas librerías, midiendo el tiempo de cada etapa sobre el dataset completo (5.8M filas), con la caché de disco precalentada para una comparación justa. Entorno: 12 núcleos lógicos, dataset de ~592 MB.

Etap	Polars (s)	Pandas (s)	Speedup
Lectura	1.419	21.232	14.97x
Filtrado	0.683	3.485	5.10x
Feature engineering	0.082	0.590	7.16x
Joins	2.249	1.797	0.80x
Agregación	0.667	2.596	3.89x
TOTAL	5.100	29.699	5.82x

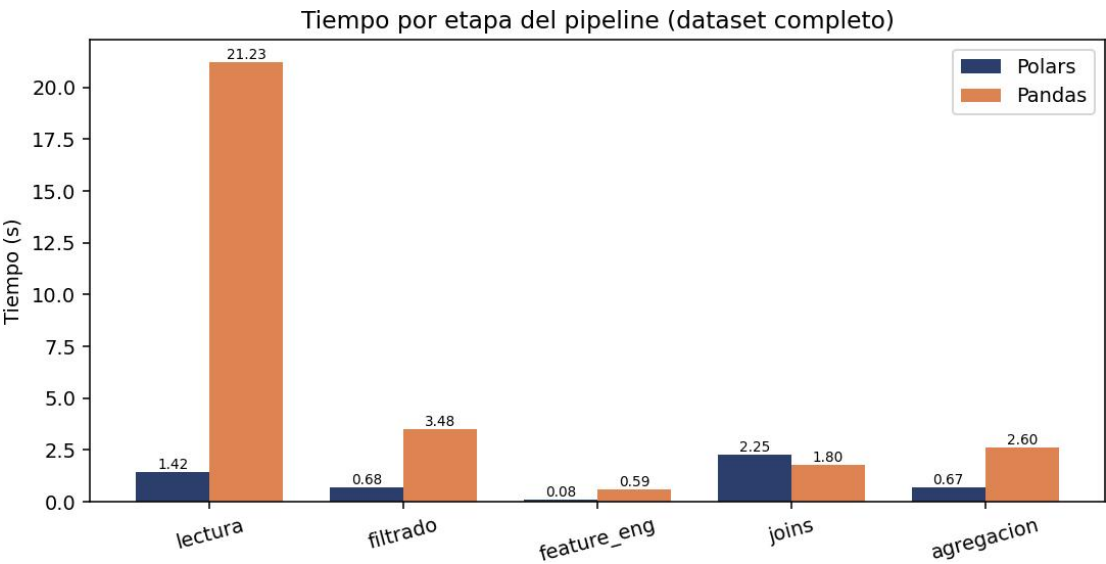


Figura 2. Tiempo por etapa

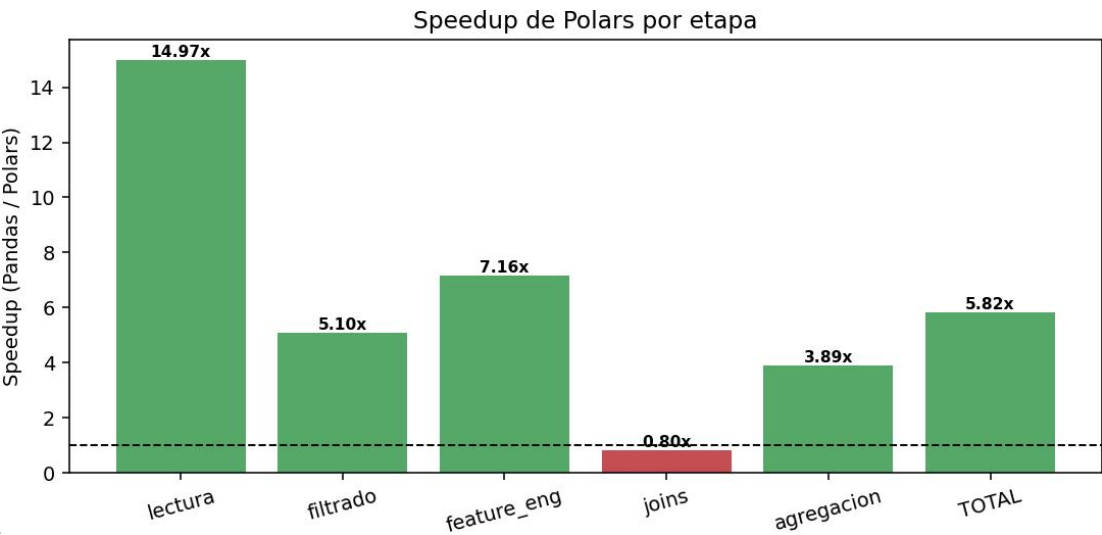


Figura 3. Speedup de Polars por etapa.

Polars fue **5.82x más rápido** en el pipeline completo. La mayor ganancia se dio en la **lectura de CSV** (14.97x), seguida del feature engineering vectorizado (7.16x) y el filtrado (5.10x). El único caso donde Pandas fue más rápido fue el **join con tablas pequeñas** (airlines.csv, 0.80x), donde el overhead de paralelización de Polars no se amortiza.

Experimento de escalabilidad

Se ejecutó el pipeline completo sobre subconjuntos del 25%, 50%, 75% y 100% del dataset, midiendo el speedup de Polars frente a Pandas en cada caso.

Porcentaje	Filas	Polars (s)	Pandas (s)	Speedup
25%	1,454,769	0.238	1.301	5.47x
50%	2,909,539	0.194	2.342	12.07x
75%	4,364,309	0.272	4.250	15.60x
100%	5,819,079	0.316	4.971	15.72x

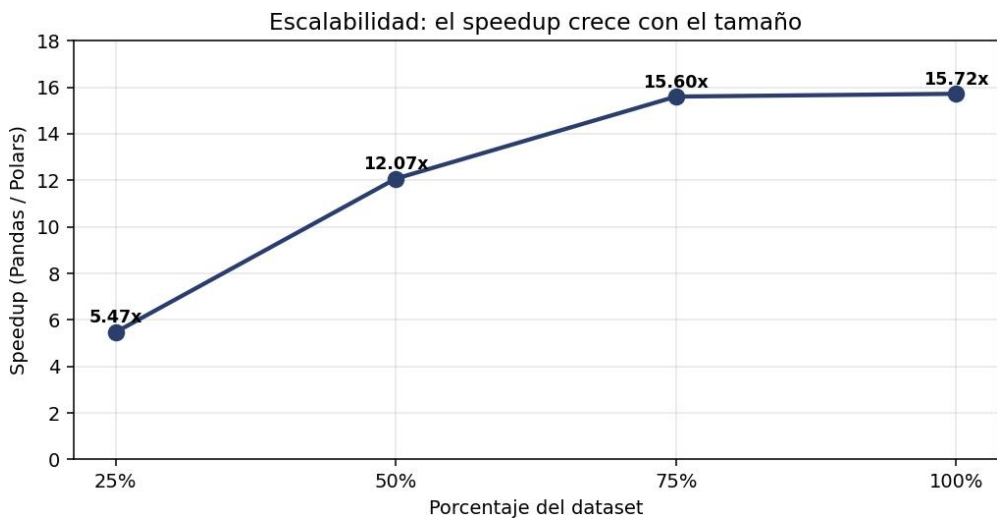


Figura 4. Speedup según el tamaño del dataset.

El **speedup crece con el tamaño** del dataset, de 5.47x (25%) a 15.72x (100%), con un promedio de 12.21x. Cuanto mayor el volumen de datos, mejor se amortizan el paralelismo multinúcleo y la arquitectura columnar de Polars. La conclusión es clara: Polars es más beneficioso cuanto más grande es el problema.

Lazy Execution (Eager vs Lazy)

Se comparó la lectura y transformación tradicional (**read_csv**, eager) contra la evaluación diferida (**scan_csv().collect()**, lazy) sobre el pipeline completo, midiendo tiempo total y memoria pico.

Modo	Tiempo total (s)	Memoria pico (MB)
Eager (read_csv)	11.768	3934.3
Lazy (scan_csv)	5.926	1429.6

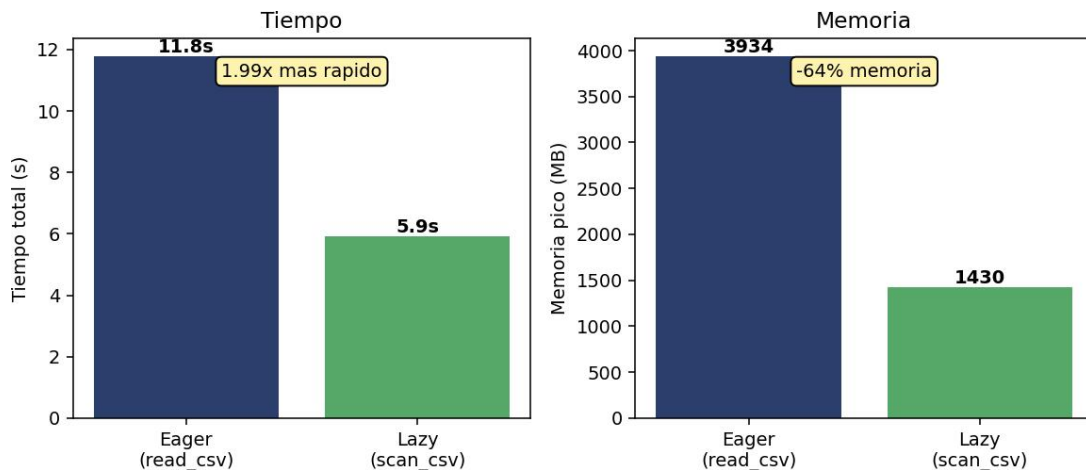


Figura 5. Eager vs Lazy: tiempo y memoria.

La ejecución lazy fue $\sim 2\times$ más rápida (11.77s vs 5.93s) y consumió $\sim 64\%$ menos memoria pico (3934 MB vs 1430 MB). El optimizador de Polars aplica projection pushdown (lee solo las columnas necesarias) y predicate pushdown (filtra durante el escaneo), evitando materializar datos que luego se descartan. Esto permite procesar datasets que no cabrían en memoria con el enfoque eager.

Análisis de Resultados

1. ¿Qué ventajas observó al utilizar Polars?

Mayor velocidad en todo el pipeline (5.82 $\times$ total, hasta 14.97 $\times$ en lectura), menor consumo de memoria (lazy usó $\sim 64\%$ menos), paralelización multinúcleo automática, API de expresiones vectorizadas concisa y evaluación diferida con optimización del plan.

2. ¿Qué operaciones obtuvieron el mayor speedup?

La lectura de CSV (14.97 $\times$), el feature engineering vectorizado (7.16 $\times$) y el filtrado (5.10 $\times$): operaciones intensivas en I/O y cómputo columnar.

3. ¿En cuáles operaciones la diferencia fue pequeña?

En los joins con tablas pequeñas (airlines.csv), donde Pandas fue incluso un poco más rápido (0.80 $\times$): el overhead paralelo de Polars no se amortiza con tablas chicas.

4. ¿Qué beneficios aporta Lazy Execution?

Sobre el dataset completo, fue $\sim 2\times$ más rápida y usó $\sim 64\%$ menos memoria que la versión eager, gracias a projection y predicate pushdown. Permite procesar datos que no caben en memoria.

5. ¿Qué limitaciones encontró en Polars?

Ecosistema más nuevo y menos integrado con librerías que esperan pandas/numpy (scikit-learn, matplotlib), lo que obliga a conversiones; sin ventaja en tablas pequeñas; curva de aprendizaje de la API; y diferencias en el manejo de nulos.

6. ¿Qué ventajas mantiene Pandas?

Ecosistema maduro, integración nativa con todo el stack de ciencia de datos, abundante documentación, mejor desempeño relativo en datasets pequeños y una API ampliamente conocida.

7. ¿La aceleración observada justifica migrar un proyecto existente?

Depende: para pipelines con datasets grandes (>1M filas) y operaciones pesadas de I/O y agregación, los 5–15 $\times$ lo justifican. Para proyectos pequeños o muy acoplados a Pandas, puede no compensar. Recomendable migrar

primero los cuellos de botella.

8. ¿Cómo afecta el tamaño del dataset al beneficio obtenido?

El speedup crece con el tamaño: de 5.47× (25%) a 15.72× (100%). Cuanto más grande el dataset, mayor el beneficio de Polars.

9. ¿Qué modelo produjo el mejor desempeño predictivo?

Por AUC, el Gradient Boosting (HistGradientBoosting) con 0.9719. Random Forest lidera en accuracy/F1 pero es el más lento. La Regresión Logística es muy competitiva y la más rápida. DEPARTURE_DELAY fue el predictor dominante.

10. ¿Qué recomendaciones daría para proyectos futuros?

Usar Polars (modo lazy con scan_csv) para el ETL de datos grandes; mantener pandas/numpy solo en el borde final para interoperar con ML y visualización; preferir HistGradientBoosting o LightGBM sobre Random Forest cuando el tiempo importa; y cuidar la fuga de datos en el diseño de características.

Conclusiones

Polars demostró una ventaja de rendimiento sustancial sobre Pandas en este pipeline de datos a gran escala, alcanzando un speedup total de **5.82×** y hasta **15.72×** en el dataset completo, con un beneficio que **aumenta con el tamaño** de los datos. La evaluación diferida (lazy) añadió una reducción de **~64% en memoria** además de duplicar la velocidad, gracias a la optimización de consultas. En la tarea predictiva, los tres modelos superaron un AUC de 0.96; el **Gradient Boosting** ofreció el mejor balance entre desempeño (AUC 0.9719) y tiempo de entrenamiento. El retraso de salida resultó ser el predictor más informativo.

En conjunto, Polars es altamente recomendable para el procesamiento de datos voluminosos, mientras que Pandas conserva valor por su madurez e integración. La estrategia óptima combina ambos: Polars para el ETL pesado y el ecosistema Pandas/numpy en la etapa final de modelado.